

motor-movimentos

```
#include "../include/logica.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
```

// Função auxiliar que verifica se a regra de Vazio ('V') bate certo com o estado da pilha

```
int regra_combina_estado(int qtd_dest, char *flags) {
    int tem_v = (strchr(flags, 'V') != NULL || strchr(flags, 'v') != NULL);
    if (qtd_dest == 0 && tem_v) return 1;
    if (qtd_dest > 0 && !tem_v) return 1;
    return 0;
}
```

Verifica se ao mover para uma coluna vazia, ela tem a regra

// Função auxiliar que verifica se as cartas estão em sequência perfeita do mesmo naipe

```
int verificar_bloco(Pilha *o, int q) {
    int i = 1;
    while (i < q) {
        Carta cima = o->cartas[o->quantidade - q + i - 1];
        Carta baixo = o->cartas[o->quantidade - q + i];
        if (cima.naipe != baixo.naipe || cima.num != baixo.num + 1) return 0;
        i++;
    }
    return 1;
}
```

Verifica se q cartas estão em sequência

// Função auxiliar que decide as regras específicas para colunas vazias

```
int decidir_vazio(Carta c, Pilha *d, char *flags, int q) {
    if (strchr(flags, '0') != NULL) return 1;
    if (d->nome_tipo[0] == 'F') {
        if (q == 13) return (c.num == 13); // Simple Simon
        return (c.num == 1); // Klondike/Freecell
    }
    if (strchr(flags, 'K') != NULL || strchr(flags, 'k') != NULL) return (c.num == 13);
    return 1;
}
```

Se for uma fundação
Se for a fundação normal, existe que a carta seja o rei
Se tiver o 0, é porque apenas a pilha tem de estar vazia e deixa a carta ir
Se tiver de re o Rei a mover para a pilha vazia, reforma se ele pode mover ou não apenas se a carta for o rei

// Função que procura a regra exata para usar ao mover uma carta da pilha

```
char* encontrar_regra(EstadoJogo *e, Pilha *orig, Pilha *dest) {
    int i = 0;
    while (i < e->total_regras) {
        if (strcmp(e->regras[i].origem, orig->nome_tipo) == 0 &&
            strcmp(e->regras[i].destino, dest->nome_tipo) == 0) {
            if (regra_combina_estado(dest->quantidade, e->regras[i].flags))
                return e->regras[i].flags;
        }
        i++;
    }
    return NULL;
}
```

Se não houver diferença entre o local de onde vem a regra e o nome do destino
Se não houver diferença entre o local de origem da regra e o nome da pilha
Se a regra encontrada for a de que tem de estar vazia e o destino não está, é porque não é para aplicar essa regra e avança até encontrar a correta

Fizemos isto por causa dos jogos que têm duas regras para o mesmo movimento.

Por exemplo no Klondike ao mover para a fundação temos a regra AV (caso vazia) e M> (caso tenha cartas). Assim resolvemos o lugar de qualquer cartas poder ir para o fundamento


```
// Função que verifica se as cartas estão em sequência
int regra_sequencia(Carta c1, Carta c2, char sinal) {
    if (sinal == '<') return (c1.num == c2.num - 1);
    if (sinal == '>') return (c1.num == c2.num + 1);
    if (sinal == 'V') return (c1.num == c2.num - 1 || c1.num == c2.num + 1);
    return 1;
}
```

Verifica se as cartas formam uma sequência válida de acordo com a regra

```
// Função que verifica se as cores são diferentes
int regra_cor(Carta c1, Carta c2) {
    int cor1 = (c1.naipes == 1 || c1.naipes == 2);
    int cor2 = (c2.naipes == 1 || c2.naipes == 2);
    return (cor1 != cor2);
}
```

Se a cor for diferente devolve verdadeiro

```
// Função auxiliar que verifica apenas cor e naipes
int aplicar_regras_naipes(Carta c_o, Carta c_d, char *flags) {
    if ((strchr(flags, 'D') || strchr(flags, 'd')) && !regra_cor(c_o, c_d))
        return 0; // Se tem do rei e a cor é igual retorna 0
    if ((strchr(flags, 'M') || strchr(flags, 'm')) && c_o.naipes !=
        c_d.naipes) return 0; // Se os naipes tiverem de rei = e forem != retorna 0
    return 1;
}
```

```
// Função auxiliar que verifica apenas a direção (< e >)
int aplicar_regras_direcao(Carta c_o, Carta c_d, char *flags) {
    if (strchr(flags, '<') && !regra_sequencia(c_o, c_d, '<')) return 0;
    if (strchr(flags, '>') && !regra_sequencia(c_o, c_d, '>')) return 0;
    return 1;
}
```

Verifica se estão em sequência

```
// Função auxiliar que verifica apenas os vizinhos (V e ~)
int aplicar_regras_vizinhos(Carta c_o, Carta c_d, char *flags) {
    if (strchr(flags, '~') != NULL && !regra_sequencia(c_o, c_d, 'V'))
        return 0; // Retorna 0 se tem de rei e não são vizinhos
    return 1;
}
```

```
// Função que aplica as regras com recurso às auxiliares
int aplicar_regras(Carta c_o, Carta c_d, char *flags) {
    if (!aplicar_regras_naipes(c_o, c_d, flags)) return 0;
    if (!aplicar_regras_direcao(c_o, c_d, flags)) return 0;
    if (!aplicar_regras_vizinhos(c_o, c_d, flags)) return 0;
    return 1;
}
```

Se falhar nas regras retorna 0

```
// Função auxiliar que decide se uma carta pode ir para o vazio
int verificar_vazio(Carta c_origem, char *flags) {
    if (strchr(flags, 'A') != NULL || strchr(flags, 'a') != NULL) return
        (c_origem.num == 1); // Retorna verdadeiro se a carta for o Ás
    if (strchr(flags, 'K') != NULL || strchr(flags, 'k') != NULL) return
        (c_origem.num == 13);
    return 1;
}
```

Se o vazio tem uma carta há de ir para o vazio

// Função que usa as anteriores para verificar se um movimento é válido

```
int verificar_movimento(Pilha *o, Pilha *d, char *flags, int q) {
    if (q <= 0 || q > o->quantidade || flags == NULL) return 0; // Condição de regras
    if (q > 1 && !verificar_bloco(o, q)) return 0; // Se tentar mover mais que 1 carta e não estiver em sequência resolve
    Carta c_origem = o->cartas[o->quantidade - q];
    if (d->quantidade == 0) { // Se não há espaço
        return decidir_vazio(c_origem, d, flags, q); // Retornar se todo se movida para lá ou não
    }
    Carta c_destino = d->cartas[d->quantidade - 1]; // Carta destino
    return aplicar_regras(c_origem, c_destino, flags); // Verifica as regras
}
```

//
// Sistema de dicas
//

// Função que pega na carta de topo de uma pilha e verifica testando uma a uma qual é a pilha para a qual ela pode mover

```
int testar_destinos(EstadoJogo *e, int o, int *dica_o, int *dica_d) {
    int d = 0;
    while (d < e->total_pilhas) {
```

Para as pilhas diferentes da origem

```
    if (o != d) {
        char *flags = encontrar_regra(e, &e->pilhas[o], &e->pilhas[d]);
        if (verificar_movimento(&e->pilhas[o], &e->pilhas[d], flags, 1)) {
            *dica_o = o; *dica_d = d; // guarda as pilhas como dicas possíveis
            return 1;
        }
        d++;
    }
    return 0;
}
```

Retorno que encontrou
Testa a próxima coluna

// Função que percorre as pilhas chamando a função anterior para criar uma dica

```
int pedir_dica(EstadoJogo *e, int *dica_o, int *dica_d) {
    int o = 0;
    while (o < e->total_pilhas) {
        if (e->pilhas[o].quantidade > 0) {
            if (testar_destinos(e, o, dica_o, dica_d)) return 1;
        }
        o++;
    }
```

return 0; // acontece caso não haja + movimentos possíveis